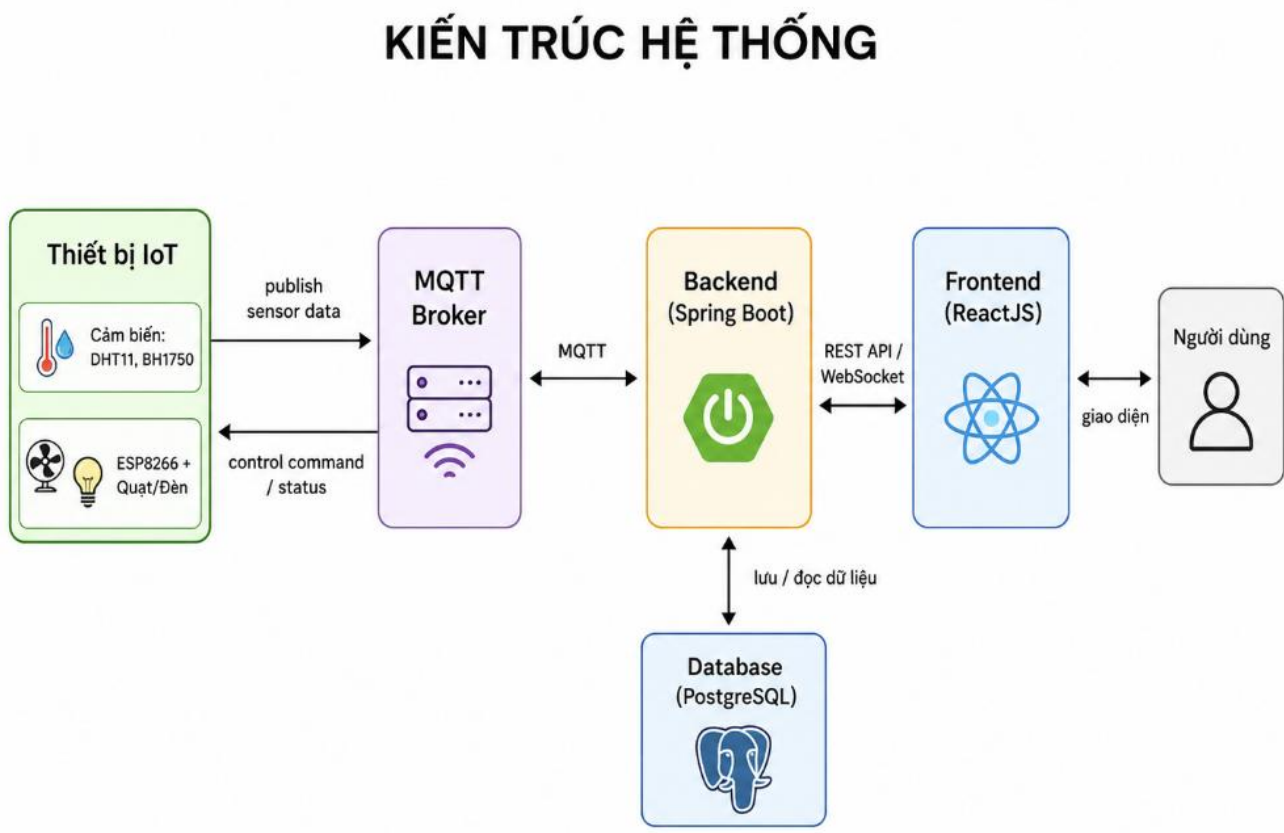
CHƯƠNG 1. THIẾT KẾ TỔNG QUAN HỆ THỐNG

1.1 Kiến trúc tổng quan

Hệ thống IoT Dashboard được thiết kế theo mô hình kiến trúc phân tầng chuẩn mực trong các hệ thống IoT công nghiệp, gồm 5 thành phần phối hợp chặt chẽ:



Hình 1 Kiến trúc tổng quan hệ thống

Hệ thống vận hành theo 2 luồng công việc chính:

1. Luồng thu thập giám sát & tự động phân loại cảnh báo: Cứ mỗi 2 giây, ESP32 đọc thông số từ cảm biến DHT11 và quang trở LDR, lấy thời gian thực từ NTP rồi đóng gói thành chuỗi JSON. Dữ liệu được Publish lên topic iot/data_sensor. Mosquitto Broker chuyển tiếp tới Spring Boot. Backend bóc tách chuỗi JSON, thông qua logic @PrePersist tự động tính toán các nhãn trạng thái cảnh báo (temp_status, humid_status, light_status), lưu bản ghi vào bảng datasensor trong MySQL, đồng thời gửi thông điệp qua WebSocket tới giao diện ReactJS để làm mới biểu đồ và thông báo cảnh báo tức thời.

2. Luồng điều khiển và xác nhận (Control & Downlink Feedback Flow): Người dùng click nút bật/tắt thiết bị trên giao diện Web. ReactJS gửi HTTP POST tới Backend. Spring Boot tạo bản ghi lịch sử ở bảng action_log với trạng thái loading và ghi nhận người thao tác user_action, sau đó Publish lệnh (ví dụ: LED1_ON) lên topic iot/devices_control. ESP32 lắng nghe topic này, kích hoạt chân GPIO đóng mở LED tương ứng, và lập tức gửi thông điệp xác nhận lên topic iot/devices_status. Backend nhận được phản hồi này, cập nhật trạng thái bản ghi sang success và thông báo tới Frontend.

1.2 Thiết kế phần cứng

1.2.1 Vi điều khiển trung tâm ESP32

Trái tim của khối phần cứng là vi điều khiển SoC ESP32-WROOM-32. Đây là dòng chip 32-bit lõi kép với tốc độ xung nhịp lên đến 240 MHz, tích hợp sẵn Wi-Fi chuẩn 802.11 b/g/n và Bluetooth v4.2 BR/EDR/BLE.

- Điện áp hoạt động: 3.3V DC (cấp nguồn qua cổng microUSB 5V và IC hạ áp AMS1117-3.3V).
- Tích hợp bộ chuyển đổi tín hiệu tương tự sang số (ADC) độ phân giải 12-bit (cho ra giá trị từ 0 đến 4095).
- Ưu điểm so với ESP8266: Bộ nhớ SRAM lớn hơn (520 KB so với 80 KB), tốc độ xử lý vượt trội, nhiều chân GPIO độc lập hơn, không bị treo khi vừa xử lý kết nối Wi-Fi vừa đọc cảm biến liên tục.

1.2.2 Khối cảm biến thu thập thông số môi trường

Cảm biến nhiệt độ và độ ẩm DHT11:

- Dải đo nhiệt độ: 0°C đến 50°C (sai số ±2°C).
- Dải đo độ ẩm: 20% đến 90% RH (sai số ±5% RH).
- Chuẩn giao tiếp: Digital Single-Bus (1-Wire độc quyền), truyền dữ liệu nối tiếp theo dạng xung thời gian, yêu cầu vi điều khiển đọc chính xác 40 bit dữ liệu (16 bit độ ẩm, 16 bit nhiệt độ, 8 bit checksum).

Cảm biến quang trở LDR:

- Sử dụng quang trở cadmium sulfide (CdS) phối hợp với điện trở kéo xuống (pull-down 10kΩ) tạo thành mạch phân áp (Voltage Divider).
- Khi cường độ ánh sáng chiếu vào tăng, nội trở LDR giảm mạnh, làm tăng điện áp đưa vào chân đọc tương tự ADC của ESP32.
- Chân GPIO34 (ADC1_CHANNEL_6) thu thập giá trị điện áp số hóa trong khoảng từ 0 (tối hoàn toàn) đến 4095 (sáng cực đại).

1.2.3 Khối cơ cấu chấp hành

Khối chấp hành bao gồm 3 đèn LED siêu sáng đại diện cho 3 tải tiêu thụ độc lập trong nhà thông minh:

- LED 1: Đại diện cho hệ thống Đèn phòng khách ("LED 01", location: "Phong Khach").
- LED 2: Đại diện cho hệ thống Đèn bếp / Quạt thông gió ("LED 02", location: "Bep").
- LED 3: Đại diện cho hệ thống Đèn phòng ngủ ("LED 03", location: "Phong Ngu").

Mỗi LED được mắc nối tiếp với một điện trở hạn dòng 220Ω hoặc 330Ω nối về chân âm (GND) nhằm bảo vệ chân vi điều khiển không bị quá tải dòng điện.

1.2.4 Sơ đồ đấu nối chân phần cứng

Bảng phân bổ chân kết nối trên ESP32:

Thiết bị / Linh kiện	Chân trên linh kiện	Chân kết nối ESP32	Chế độ cấu hình GPIO	Chức năng chi tiết
Cảm biến DHT11	VCC	3V3 / 5V	Nguồn	Cấp nguồn hoạt động cho cảm biến
	GND	GND	Nối đất	Chân mass chung
	DATA	GPIO 4	INPUT_PULLUP	Tín hiệu số 1-Wire đọc nhiệt độ, độ ẩm
Quang trở LDR	Cực 1	3V3	Nguồn	Đầu vào cầu phân áp
	Cực 2	GPIO 34	INPUT (ADC1)	Đọc điện áp tương tự tỷ lệ theo ánh sáng
	(Qua trở 10k)	GND	Nối đất	Điện trở thoát mass (Pull-down)
LED 1 (Đỏ)	Anode (+)	GPIO 5	OUTPUT (Digital)	Lệnh điều khiển Đèn 1 (HIGH=Bật, LOW=Tắt)
	Cathode (-)	GND (qua trở 220Ω)	Nối đất	Hạn dòng và thoát mass
LED 2 (Vàng)	Anode (+)	GPIO 18	OUTPUT (Digital)	Lệnh điều khiển Đèn 2 (HIGH=Bật, LOW=Tắt)

	Cathode (-)	GND (qua trở 220Ω)	Nối đất	Hạn dòng và thoát mass
LED 3 (Xanh)	Anode (+)	GPIO 19	OUTPUT (Digital)	Lệnh điều khiển Đèn 3 (HIGH=Bật, LOW=Tắt)
	Cathode (-)	GND (qua trở 220Ω)	Nối đất	Hạn dòng và thoát mass

1.2.5 Cơ chế đồng bộ thời gian thực qua giao thức NTP

Trong các hệ thống giám sát viễn thông, tính chính xác về mốc thời gian (timestamp) của dữ liệu là yếu tố sống còn. Để tránh việc gói tin lưu trong cơ sở dữ liệu bị sai lệch giờ hoặc phụ thuộc vào module RTC rời đắt đỏ, vi điều khiển ESP32 được cấu hình đồng bộ trực tiếp với máy chủ thời gian thế giới pool.ntp.org:

- Máy chủ NTP: pool.ntp.org
- Độ lệch múi giờ (GMT+7): $7 \times 3600 = 25200$ giây.
- Định dạng thời gian chuẩn hóa: yyyy-MM-dd HH:mm:ss.
- Hàm configTime(gmtOffset_sec, daylightOffset_sec, ntpServer) được gọi ngay trong hàm setupWiFi(). Thiết bị sẽ chờ cho đến khi hàm getLocalTime(&timeinfo) đồng bộ thành công mới chính thức gửi các gói tin dữ liệu, loại bỏ hoàn toàn hiện tượng bản ghi bị gán nhãn N/A.

1.3 Giao thức truyền thông mạng và Broker MQTT

1.3.1 Mô hình Publish/Subscribe

MQTT là giao thức truyền thông điệp gọn nhẹ hoạt động trên nền TCP/IP, được tối ưu hóa đặc biệt cho các thiết bị hạn chế về năng lượng và băng thông. Khác với giao thức HTTP yêu cầu mô hình Request-Response liên tục giữ kết nối nặng nề, MQTT tách rời thiết bị gửi (Publisher) và thiết bị nhận (Subscriber) thông qua một máy chủ trung chuyển gọi là Broker.

1.3.2 Cấu hình xác thực và bảo mật Mosquitto Broker

Hệ thống sử dụng phần mềm mã nguồn mở Eclipse Mosquitto chạy cục bộ:

- Cổng dịch vụ (Port): 2205 (cấu hình trong application.properties và code.ino).
- Cơ chế bảo mật: Bật cấm truy cập ẩn danh (allow_anonymous false), yêu cầu xác thực bằng tập tin mật khẩu:

- Username: anhtran
- Password: 220505

- Cấu hình này ngăn chặn các truy cập trái phép gửi lệnh phá hoại hoặc đọc trộm dữ liệu môi trường.

1.3.3 Đặc tả Topics và định dạng Payload JSON

Tên Topic	Hướng truyền dữ liệu	Định dạng dữ liệu	Mục đích và ý nghĩa
iot/data_sensor	ESP32 → Backend	JSON Object	Gửi dữ liệu đo đạc cảm biến định kỳ mỗi 2 giây
iot/devices_control	Backend → ESP32	Plain Text / JSON	Gửi lệnh bật/tắt thiết bị chấp hành
iot/devices_status	ESP32 → Backend	JSON Object	Phản hồi xác nhận trạng thái thực tế của thiết bị

Chi tiết cấu trúc Payload JSON:

1. **Topic iot/data_sensor (Telemetry):**

```
{
  "timestamp": "2026-09-30 14:30:00",
  "temperature": 28.5,
  "humidity": 75.0,
  "light": 1250
}
```

2. **Topic iot/devices_control (Command):**

Bật/Tắt từng LED: "LED1_ON", “LED2_ON”, “LED3_ON”, “LED1_OFF”, “LED2_OFF”, “LED3_OFF”

Bật/Tắt toàn bộ: "ALL_ON", "ALL_OFF".

3. **Topic iot/devices_status (Status Feedback):**

```
{
  "device": "LED1",
  "status": "ON",
  "timestamp": "2026-09-30 14:30:02"
}
```

1.4 Thiết kế Cơ sở dữ liệu

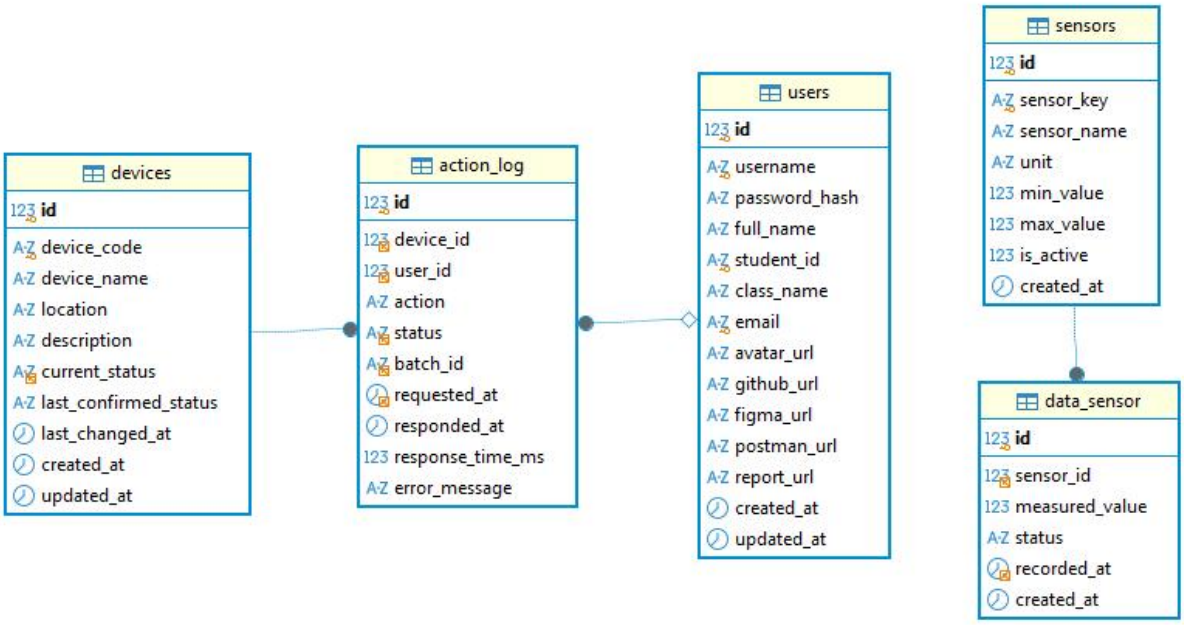
1.4.1 Tổng quan

Cơ sở dữ liệu của hệ thống IoT có nhiệm vụ:

- Lưu thông tin người dùng (xác thực, đăng nhập).
- Quản lý danh mục cảm biến và thiết bị.
- Lưu trữ dữ liệu đo được từ cảm biến theo thời gian thực.
- Ghi lại lịch sử các hành động điều khiển thiết bị.

Hệ quản trị CSDL: MySQL

1.4.2 Chi tiết các bảng



Hình 2 ERD

1.4.2.1 Bảng users – Thông tin người dùng và hồ sơ sinh viên

- Entity Java: com.ptit.iot.model.User
- Vai trò: Lưu trữ thông tin tài khoản đăng nhập và dữ liệu hiển thị trang cá nhân (Profile).

Tên trường (DB)	Thuộc tính (Entity)	Kiểu dữ liệu	Ràng buộc	Nullable	Mô tả ý nghĩa
id	id	BIGINT	PK, AI	NO	Khóa chính tự động tăng
username	username	VARCHAR(50)	UNIQUE	NO	Tên tài khoản

					đăng nhập
password	password	VARCHAR(255)	—	NO	Chuỗi băm mật khẩu bảo mật BCrypt
full_name	fullName	VARCHAR(100)	—	YES	Họ tên đầy đủ sinh viên
student_id	studentId	VARCHAR(20)	—	YES	Mã sinh viên
class_room	classRoom	VARCHAR(50)	—	YES	Lớp sinh viên
email	email	VARCHAR(100)	—	YES	Địa chỉ email
avatar_url	avatarUrl	VARCHAR(500)	—	YES	Đường dẫn ảnh chân dung đại diện
github_url	githubUrl	VARCHAR(500)	—	YES	Liên kết kho lưu trữ GitHub
figma_url	figmaUrl	VARCHAR(500)	—	YES	Liên kết tài liệu thiết kế Figma
postman_url	postmanUrl	VARCHAR(500)	—	YES	Liên kết tài liệu kiểm thử Postman API
report_url	reportUrl	VARCHAR(500)	—	YES	Liên kết file báo cáo đồ án
created_at	createdAt	DATETIME	—	NO	Thời điểm tài khoản được khởi tạo

1.4.2.2 Bảng sensor – Định nghĩa loại cảm biến vật lý trong hệ thống

- Entity Java: com.ptit.iot.model.Sensor
- Vai trò: Lưu thông tin về loại/nhóm cảm biến vật lý. Mỗi node phần cứng (ESP32+DHT11+LDR) tương ứng với 1 bản ghi trong bảng này.
- Quan hệ: 1–N với datasensor: một Sensor có nhiều bản ghi DataSensor (@OneToMany(mappedBy="sensor")). Phía DataSensor giữ FK sensor_id.

Tên trường (DB)	Thuộc tính (Entity)	Kiểu dữ liệu	Ràng buộc	Nullable	Mô tả ý nghĩa
id	id	BIGINT	PK, AI	NO	Khóa chính tự động tăng
sensor_key	sensorKey	VARCHAR(50)	UNIQUE	NO	Khóa định danh kỹ thuật
name	name	VARCHAR(100)	—	NO	Tên hiển thị
type	type	VARCHAR(50)	—	NO	Loại cảm biến
location	location	VARCHAR(100)	—	YES	Vị trí đặt cảm biến
active	active	BOOLEAN	—	NO	Trạng thái hoạt động

1.4.2.3 Bảng devices – Danh mục và trạng thái thiết bị

- Entity Java: com.ptit.iot.model.Device
- Vai trò: Quản lý danh mục 3 đèn LED và lưu giữ trạng thái hoạt động thực tế.

Tên trường (DB)	Thuộc tính (Entity)	Kiểu dữ liệu	Ràng buộc	Nullable	Mô tả ý nghĩa
id	id	BIGINT	PK, AI	NO	Khóa chính tự động tăng
device_key	deviceKey	VARCHAR(50)	UNIQUE	NO	Khóa định danh thiết bị
name	name	VARCHAR(100)	—	NO	Tên hiển thị
type	type	VARCHAR(50)	—	NO	Loại thiết

					bị
status	status	BOOLEAN	—	NO	Trạng thái bật/tắt vật lý
online	online	BOOLEAN	—	NO	Trạng thái trực tuyến của thiết bị
location	location	VARCHAR(100)	—	YES	Vị trí lắp đặt
last_changed	lastChanged	DATETIME	—	YES	Thời điểm trạng thái thay đổi gần nhất

1.4.2.4 Bảng datasensor – Dữ liệu đo đạc cảm biến và trạng thái cảnh báo

- Entity Java: com.ptit.iot.model.DataSensor
- Vai trò: Lưu trữ toàn bộ chuỗi số liệu đo lường môi trường cùng trạng thái phân loại tự động phục vụ vẽ biểu đồ và phân tích lịch sử cảnh báo.

Tên trường (DB)	Thuộc tính (Entity)	Kiểu dữ liệu	Ràng buộc	Nullable	Mô tả ý nghĩa
id	id	BIGINT	PK, AI	NO	Khóa chính tự động tăng
sensor_id	sensor	BIGINT	FK	YES	FK tới sensor.id — xác định bản ghi thuộc cảm biến nào
temperature	temperature	DOUBLE	—	NO	Nhiệt độ đo được
humidity	humidity	DOUBLE	—	NO	Độ ẩm không khí đo được
light	light	INT	—	NO	Cường độ ánh sáng

timestamp	timestamp	DATETIME	INDEX	NO	Thời điểm lấy mẫu dữ liệu từ cảm biến
temp_status	tempStatus	VARCHAR(50)	—	YES	Trạng thái nhiệt độ
humid_status	humidStatus	VARCHAR(50)	—	YES	Trạng thái độ ẩm
light_status	lightStatus	VARCHAR(50)	—	YES	Trạng thái ánh sáng

1.4.2.5 Bảng action_log – Lịch sử thao tác điều khiển thiết bị

- Entity Java: com.ptit.iot.model.ActionLog
- Vai trò: Ghi nhận nhật ký mỗi lần người dùng thao tác bật hoặc tắt thiết bị. Sử dụng quan hệ FK thực sự thay vì lưu chuỗi String lỏng lẻo như phiên bản trước.
- Quan hệ: @ManyToOne → User (qua user id), @ManyToOne → Device (qua device id).
- Serialize: Dùng ActionLogDTO (flatten) để tránh vòng lặp JSON khi trả về client.

Tên trường (DB)	Thuộc tính (Entity)	Kiểu dữ liệu	Ràng buộc	Nullable	Mô tả ý nghĩa
id	id	BIGINT	PK, AI	NO	Khóa chính tự động tăng
user_id	user	BIGINT	FK	YES	FK tới users.id — người thực hiện hành động (null nếu do hệ thống)
device_id	device	BIGINT	FK	YES	FK tới devices.id — thiết bị bị điều khiển
action	action	VARCHAR(20)	—	NO	Hành động yêu cầu: ON hoặc OFF
status	status	VARCHAR(20)	—	NO	Trạng thái thực thi: success, loading, failed
timestamp	timestamp	DATETIME	—	NO	Thời điểm gửi lệnh điều khiển

1.5 Thiết kế kiến trúc phần mềm Backend

Ứng dụng máy chủ được xây dựng trên nền tảng Java 17 và Spring Boot 3, tuân thủ mô hình phân lớp chuẩn công nghiệp (Layered Architecture):

1. Controller Layer (com.ptit.iot.controller):

- AuthController: Xử lý đăng nhập (/api/auth/login), lấy thông tin profile (GET /api/auth/profile/{username}) và cập nhật profile (PUT /api/auth/profile/{id}).
- SensorController: Cung cấp API truy xuất số liệu đo cảm biến mới nhất (/api/sensor/latest), biểu đồ (/api/sensor/chart), và danh sách phân trang với bộ lọc linh hoạt (/api/sensor).
- DeviceController: Lấy danh sách thiết bị (GET /api/devices), tiếp nhận lệnh điều khiển (POST /api/devices/control) và kiểm tra trạng thái MQTT (GET /api/devices/mqtt-status). DeviceControlRequest nhận 2 trường: command (lệnh điều khiển) và user (người thao tác — ưu tiên từ body, fallback sang HTTP Session nếu null).
- HistoryController: Cung cấp API tìm kiếm, lọc và phân trang lịch sử thao tác (/api/history).

2. Service Layer (com.ptit.iot.service):

- MqttService: Khởi tạo Eclipse Paho MQTT Client, subscribe 2 topic (iot/data_sensor, iot/devices_status). Khi nhận cảm biến: parse SensorPayload → tra cứu/tạo Sensor entity theo key DHT11_LDR → tạo DataSensor với FK sensor → @PrePersist tự động tính trạng thái → lưu DB → broadcast /topic/sensor. Khi nhận phản hồi thiết bị: parse DeviceStatusPayload → cập nhật Device.status → broadcast /topic/device-status; tìm ActionLog loading → cập nhật success → broadcast ActionLogDTO (không broadcast Entity thô để tránh lazy loading exception) lên /topic/action-log.
- DeviceService: Dispatch lệnh điều khiển. Lệnh đơn (vd: LED1_ON): tách deviceKey+action → tra cứu Device và User bằng FK thực sự → tạo ActionLog với device và user là FK object → lưu DB → broadcast ActionLogDTO.from(saved). Lệnh ALL_ON/ALL_OFF: tạo 3 bản ghi ActionLog (LED1/LED2/LED3) rồi publish 1 lần MQTT. Trả về ActionLogDTO thay vì Entity thô. Hỗ trợ overload sendControl(command) tương thích ngược.
- SensorService: Cung cấp các hàm getLatest(), getChartData(hours), và getSensorData(...) tích hợp phân trang và lọc theo khoảng giá trị min/max.

3. Repository Layer (com.ptit.iot.repository):

- DataSensorRepository: Cung cấp findWithFilters(...) (JPQL, LEFT JOIN FETCH d.sensor), findByTempStatusIsNull() (migration status), findBySensorIsNull() (migration FK).
- ActionLogRepository: findWithFilters(...) (JPQL với LEFT JOIN FETCH cả device và user), findTopByDeviceKeyAndStatusOrderByTimestampDesc() (tìm log loading chờ xác nhận).
- SensorRepository: findBySensorKey(String) — tra cứu cảm biến theo key kỹ thuật.
- DeviceRepository, UserRepository: Quản lý thực thể tương ứng qua Spring Data JPA.

4. Realtime WebSocket & Security Layer (com.ptit.iot.config):

- WebSocketConfig: Cấu hình STOMP endpoint /ws với SockJS fallback, destination prefix /app, broker prefix /topic.
- 3 broadcast topic thời gian thực: /topic/sensor (dữ liệu cảm biến), /topic/device-status (trạng thái thiết bị), /topic/action-log (cập nhật lịch sử action).
- SecurityConfig: Mở toàn bộ request (anyRequest().permitAll()), disable CSRF, CORS cho phép mọi origin (allowedOriginPatterns("*")), allowCredentials(true).
- Mật khẩu mã hóa bằng BCryptPasswordEncoder.

1.6 Thiết kế kiến trúc phần mềm Frontend

Giao diện người dùng được xây dựng bằng ReactJS 18 phối hợp với TypeScript:

- Công nghệ: ReactJS 18 + TypeScript + Vite, HTTP qua fetch với credentials: 'include' (cookie session), WebSocket qua @stomp/stompjs + sockjs-client.
- Quản lý trạng thái: useState/useCallback/useEffect trong App.tsx. Sau đăng nhập: gọi loadDevices(), loadHistory() và connectWebSocket() đồng thời.
- WebSocket: App.tsx khởi tạo STOMP Client, subscribe 3 topic: /topic/sensor → cập nhật liveSensor; /topic/device-status → cập nhật devices[]; /topic/action-log → thêm log mới vào đầu mảng (giới hạn 50 phần tử). Auto-reconnect sau 5 giây.
- Optimistic UI: Click bật/tắt thiết bị → cập nhật UI ngay → gọi API → nếu lỗi thì revert + hiển thị Toast.

- DataMapper: mapDevice(DeviceRecord→Device), mapLog(ActionLog Record→DeviceActionLog), định dạng thời gian toLocaleString('vi-VN').
- Giao diện: Nền hồng nhạt #fff8f8, text #22191d, layout responsive (Sidebar trên desktop, BottomNavBar trên mobile), các màn hình chuyển đổi không tải lại trang.
- Tổ chức Component:
    - OverviewView.tsx: Màn hình Dashboard tổng quan gồm 3 thẻ cảm biến đo lường và biểu đồ động.
    - DeviceControlView.tsx: Khu vực điều khiển 3 bóng đèn LED với các nút gạt tương tác trực quan và spinner báo trạng thái đang xử lý.
    - SensorHistoryView.tsx: Bảng tra cứu lịch sử cảm biến với thanh tìm kiếm đa năng (SearchMode), hỗ trợ phân tích dải giá trị parseValueRange và bộ lọc thời gian linh hoạt parseFlexibleDateTime.
    - DeviceHistoryView.tsx: Bảng nhật ký điều khiển thiết bị chi tiết với các cột: thiết bị (deviceName, deviceKey), hành động (ON/OFF), trạng thái (success/loading/failed) và người thao tác (user). Hỗ trợ lọc theo thiết bị, hành động, trạng thái và khoảng thời gian.
    - ProfileView.tsx: Trang giới thiệu thông tin cá nhân của sinh viên Phạm Mai Linh, hình đại diện và các liên kết dự án liên quan (GitHub, Figma, Postman, Báo cáo).
    - LoginScreen.tsx: Màn hình đăng nhập xác thực tài khoản.

1.7 API DOCUMENTATION

- Base URL: http://localhost:8080/api
- Authentication: JWT Bearer Token - truyền qua header Authorization: Bearer <token>
- Content-Type: application/json

STT	Phương thức	Endpoint URI	Mô tả chức năng	Quyền truy cập
1	POST	/api/auth/login	Đăng nhập hệ thống, trả JWT và thông tin người dùng	Public
2	GET	/api/sensor/latest	Lấy giá trị đo và trạng thái mới nhất của cảm biến	Authenticated
3	GET	/api/sensor/chart	Lấy chuỗi điểm dữ liệu phục vụ vẽ biểu đồ (?hours=1)	Authenticated
4	GET	/api/sensor	Tra cứu lịch sử cảm biến (Phân trang, Lọc dải giá trị min/max, thời gian)	Authenticated
5	GET	/api/devices	Lấy danh mục và trạng thái hiện tại của 3 thiết bị LED	Authenticated
6	POST	/api/devices/control	Gửi lệnh bật/tắt thiết bị chấp hành ({"command": "LED1_ON"})	Authenticated
7	GET	/api/devices/mqtt-status	Kiểm tra tình trạng kết nối tới MQTT Broker	Authenticated
8	GET	/api/history	Tra cứu lịch sử điều khiển bật/tắt (Lọc đa chiều, phân trang)	Authenticated
9	WS	/ws	Kênh WebSocket truyền dữ liệu cảm biến & thiết bị thời gian thực	Public / Client

10	GET	/api/auth/profile/{username}	Truy xuất thông tin Profile cá nhân của người dùng	Authenticated
11	PUT	/api/auth/profile/{id}	Cập nhật hồ sơ và các URL liên kết (GitHub, Figma, v.v)	Authenticated
12	POST	/api/auth/profile/{id}/avatar	Tải ảnh đại diện PNG/JPG	Authenticated

1.7.1 API-01 - Đăng nhập

Thuộc tính	Nội dung
Method	POST
Endpoint	/api/auth/login
Mô tả	Xác thực người dùng bằng username/password. Trả về thông tin User để Frontend lưu phiên làm việc (Session).
Auth	Không cần
Bảng DB liên quan	users

Request Body:

```
{
  "username": "admin",
  "password": "admin123"
}
```

Response Body (200 OK):

```
{
  "success": true,
  "message": "Đăng nhập thành công",
  "data": {
    "id": 1,
    "username": "admin",
    "fullName": "Trần Ngọc Ánh",
    "email": "trananh22052005@gmail.com",
    "studentId": "B23DCAT022",
    "classRoom": "D23CQAT02-B",
    "githubUrl": "https://github.com/...",
    "figmaUrl": "https://figma.com/...",
    "postmanUrl": "https://postman.com/...",
```

```
    "reportUrl": "https://drive.google.com/..."
  }
}
```

HTTP Status:

Code	Ý nghĩa
200	Đăng nhập thành công
400	Thiếu username hoặc password
401	Username hoặc mật khẩu không đúng
500	Lỗi server

1.7.2 API-02 - Lấy giá trị cảm biến mới nhất

Thuộc tính	Nội dung
Method	GET
Endpoint	/api/sensor/latest
Mô tả	Lấy bản ghi đo lường gần nhất của cả 3 chỉ số (nhiệt độ, độ ẩm, ánh sáng) kèm trạng thái cảnh báo (được tính bởi @PrePersist) để hiển thị thẻ số liệu trên Dashboard.
Auth	Không bắt buộc (Session)
Bảng DB liên quan	datasensor

Request Parameters: Không có

Response Body (200 OK):

```
{
  "success": true,
  "code": "IOT-000",
  "message": "Thành công",
  "data": {
    "temperature": { "value": 28.5, "unit": "°C", "status": "NORMAL", "recordedAt": "2026-10-20T20:00:00+07:00" },
    "humidity": { "value": 60, "unit": "%", "status": "NORMAL", "recordedAt": "2026-10-20T20:00:00+07:00" },
    "light": { "value": 300, "unit": "lux", "status": "NORMAL", "recordedAt": "2026-10-20T20:00:00+07:00" },
    "lastUpdatedAt": "2026-10-20T20:00:00+07:00"
  }
```

```
}
```

HTTP Status:

Code	Ý nghĩa
200	Thành công (trả về null nếu chưa có dữ liệu)
404	Chưa có bất kỳ bản ghi cảm biến nào trong DB
500	Lỗi server

1.7.3 API-03 - Lấy dữ liệu vẽ biểu đồ

Thuộc tính	Nội dung
Method	GET
Endpoint	/api/sensor/chart
Mô tả	Lấy chuỗi điểm dữ liệu lịch sử để vẽ biểu đồ đường (Line Chart) với 3 series: nhiệt độ, độ ẩm, ánh sáng. Lọc theo số giờ gần đây.
Auth	Không bắt buộc (Session)
Bảng DB liên quan	datasensor

Query Parameters:

Param	Type	Bắt buộc	Mô tả
hours	integer	Không	Số giờ gần đây cần lấy dữ liệu. Mặc định: 1

Ví dụ Request: GET /api/sensor/chart?hours=1

Response Body (200 OK):


```
{
  "success": true,
  "message": "Lấy dữ liệu biểu đồ thành công",
  "data": [
    {
      "id": 1040,
      "temperature": 27.5,
      "humidity": 73.0,
      "light": 980,
      "timestamp": "2026-09-30T13:30:00",
      "tempStatus": "Bình thường",
```

```
      "humidStatus": "Cảnh báo cao",
      "lightStatus": "Bình thường"
    },
    {
      "id": 1041,
      "temperature": 28.0,
      "humidity": 74.5,
      "light": 1050,
      "timestamp": "2026-09-30T13:30:02",
      "tempStatus": "Bình thường",
      "humidStatus": "Cảnh báo cao",
      "lightStatus": "Cảnh báo cao"
    }
  ]
}
```


HTTP Status:

Code	Ý nghĩa
200	Thành công (có thể trả về mảng rỗng)
400	Tham số hours không hợp lệ (âm hoặc không phải số)
500	Lỗi server

1.7.4 API-04 - Tra cứu lịch sử dữ liệu cảm biến

Thuộc tính	Nội dung
Method	GET
Endpoint	/api/sensor
Mô tả	Lấy danh sách lịch sử dữ liệu cảm biến theo trang. Hỗ trợ lọc theo khoảng thời gian from/to, lọc theo dải giá trị min/max của từng chỉ số. Câu truy vấn JPQL động DataSensorRepository.findWithFilters() xử lý tất cả tham số tùy chọn. Sắp xếp theo timestamp giảm dần.
Auth	Không bắt buộc (Session)
Bảng DB liên quan	datasensor

Query Parameters:

Param	Type	Bắt buộc	Mô tả

from	string	Không	Thời gian bắt đầu — ISO LocalDateTime (vd: 2026-09-30T00:00:00)
to	string	Không	Thời gian kết thúc — ISO LocalDateTime
minTemp	number	Không	Ngưỡng nhiệt độ tối thiểu (°C)
maxTemp	number	Không	Ngưỡng nhiệt độ tối đa (°C)
minHumid	number	Không	Ngưỡng độ ẩm tối thiểu (%)
maxHumid	number	Không	Ngưỡng độ ẩm tối đa (%)
minLight	integer	Không	Ngưỡng ánh sáng tối thiểu (lux)
maxLight	integer	Không	Ngưỡng ánh sáng tối đa (lux)
page	integer	Không	Trang hiện tại (bắt đầu từ 0). Mặc định: 0
size	integer	Không	Số bản ghi mỗi trang. Mặc định: 10

Ví dụ request:

GET /api/sensor?minTemp=25&maxTemp=30&page=0&size=10

Response Body (200 OK):


```
{
  "success": true,
  "message": "Lấy dữ liệu thành công",
  "data": {
    "content": [
      {
        "id": 105,
        "temperature": 28.5,
        "humidity": 75.0,
        "light": 1250,
        "timestamp": "2026-09-30T14:30:00",
        "tempStatus": "Bình thường",
        "humidStatus": "Cảnh báo cao",
        "lightStatus": "Cảnh báo cao"
      }
    ],
    "totalElements": 340,
    "totalPages": 34,
    "currentPage": 0,
    "size": 10
```

```
    }
}
```

HTTP Status:

Code	Ý nghĩa
200	Thành công (có thể trả về mảng rỗng nếu không có bản ghi khớp)
400	Tham số không hợp lệ (vd: from > to, minTemp > maxTemp)
500	Lỗi server

1.7.5 API-05 - Lấy danh sách và trạng thái thiết bị

Thuộc tính	Nội dung
Method	GET
Endpoint	/api/devices
Mô tả	Lấy danh sách tất cả thiết bị trong hệ thống kèm trạng thái bật/tắt hiện tại (status: boolean), tình trạng kết nối (online: boolean) và vị trí lắp đặt.
Auth	Không bắt buộc (Session)
Bảng DB liên quan	devices

Response Body (200 OK):

```
{
  "success": true,
  "message": "Lấy danh sách thiết bị thành công",
  "data": [
    {
      "id": 1, "deviceKey": "LED1", "name": "LED 01",
      "type": "light", "status": false, "online": true,
      "location": "Phong Khach", "lastChanged": "2026-09-30T14:25:00"
    },
    {
      "id": 2, "deviceKey": "LED2", "name": "LED 02",
      "type": "light", "status": true, "online": true,
      "location": "Bep", "lastChanged": "2026-09-30T13:00:00"
    },
```

```
        {
          "id": 3, "deviceKey": "LED3", "name": "LED 03",
          "type": "light", "status": false, "online": true,
          "location": "Phong Ngu", "lastChanged": "2026-09-
30T14:30:00"
        }
      ]
    }
```

HTTP Status:

Code	Ý nghĩa
200	Thành công
500	Lỗi server

1.7.6 API-06 - Gửi lệnh điều khiển bật/tắt thiết bị

Thuộc tính	Nội dung
Method	POST
Endpoint	/api/devices/control
Mô tả	Gửi lệnh điều khiển thiết bị. Backend Publish lệnh lên MQTT topic iot/devices_control và tạo bản ghi action_log với trạng thái loading. Khi nhận phản hồi từ ESP32 qua topic iot/devices_status, trạng thái được cập nhật thành success.
Auth	Không bắt buộc (Session)
Bảng DB liên quan	action_log, devices
MQTT	Publish tới topic: iot/devices_control — payload: "LED1_ON"

Response Body (200 OK):

```
{
  "command": "LED1_ON",
  "user": "admin"
}
```

Danh sách lệnh hợp lệ:

Lệnh	Hành động
LED1_ON / LED1_OFF	Bật / Tắt LED 1
LED2_ON / LED2_OFF	Bật / Tắt LED 2
LED3_ON / LED3_OFF	Bật / Tắt LED 3

ALL_ON / ALL_OFF	Bật / Tắt tất cả LED đồng thời

Response Body (202 Accepted):

```
{
  "success": true,
  "message": "Đã gửi lệnh điều khiển tới thiết bị",
  "data": "LED1_ON"
}
```

HTTP Status:

Code	Ý nghĩa
202	Lệnh đã được gửi đến Broker thành công (chờ ESP32 phản hồi)
400	Lệnh không hợp lệ hoặc để trống
500	Lỗi server hoặc MQTT Broker không kết nối được

1.7.7 API-07 - Kiểm tra trạng thái kết nối MQTT

Thuộc tính	Nội dung
Method	GET
Endpoint	/api/devices/mqtt-status
Mô tả	Kiểm tra tình trạng kết nối hiện tại của Spring Boot MqttService tới Mosquitto Broker, trả về connected: true/false. Dùng để Frontend hiển thị cảnh báo khi Broker mất kết nối.
Auth	Không bắt buộc

Response Body (200 OK):

```
{
  "success": true,
  "message": "Kiểm tra thành công",
  "data": {
    "connected": true
  }
}
```

HTTP Status:

Code	Ý nghĩa
200	Thành công (connected: true hoặc false)
500	Lỗi server

1.7.8 API-08 - Tra cứu lịch sử điều khiển thiết bị (Lọc đa chiều)

Thuộc tính	Nội dung
Method	GET
Endpoint	/api/history
Mô tả	Lấy danh sách lịch sử bật/tắt thiết bị theo trang. Hỗ trợ lọc theo khóa thiết bị (deviceKey), hành động (action), trạng thái (status) và khoảng thời gian from/to. Trả về ActionLogDTO (flatten từ ActionLog + User + Device) tránh vòng lặp JSON. Sắp xếp theo timestamp giảm dần.
Auth	Không bắt buộc (Session)
Bảng DB liên quan	action_log

Query Parameters:

Param	Type	Bắt buộc	Mô tả
deviceKey	string	Không	Lọc theo thiết bị: LED1, LED2, LED3
action	string	Không	Lọc theo hành động: ON hoặc OFF
status	string	Không	Lọc theo trạng thái: success, loading, failed
from	string	Không	Thời gian bắt đầu — ISO LocalDateTime
to	string	Không	Thời gian kết thúc — ISO LocalDateTime
page	integer	Không	Trang hiện tại (bắt đầu từ 0). Mặc định: 0
size	integer	Không	Số bản ghi mỗi trang. Mặc định: 10

Ví dụ Request:

GET /api/history?deviceKey=LED1&action=ON&status=success&page=0&size=10

Response Body (200 OK):


```
{
  "success": true,
  "message": "Lấy lịch sử thành công",
  "data": {
    "content": [
      {
        "id": 45,
        "deviceKey": "LED1",
        "deviceName": "LED 01",
        "deviceType": "light",
```

```
            "username": "admin",
            "userFullName": "Tran Ngoc Anh",
            "action": "ON",
            "status": "success",
            "timestamp": "2026-09-30T14:30:00"
          },
          {
            "id": 44,
            "deviceKey": "LED1",
            "deviceName": "LED 01",
            "deviceType": "light",
            "username": "admin",
            "userFullName": "Tran Ngoc Anh",
            "action": "OFF",
            "status": "success",
            "timestamp": "2026-09-30T13:45:00"
          }
        ],
        "totalElements": 120,
        "totalPages": 12,
        "currentPage": 0,
        "size": 10
      }
    }
```


HTTP Status:

Code	Ý nghĩa
200	Thành công (có thể trả về mảng rỗng)
400	Tham số không hợp lệ (vd: action không phải ON/OFF)
500	Lỗi server

1.7.9 API-09 - Kênh WebSocket/STOMP thời gian thực

Thuộc tính	Nội dung
Giao thức	WebSocket / STOMP
Endpoint kết nối	/ws
Mô tả	Kênh kết nối WebSocket hai chiều để Backend chủ động đẩy (Push) dữ liệu mới về phía Frontend ngay khi nhận được gói tin từ

	phần cứng qua MQTT. Không cần Client polling. Dùng giao thức STOMP (Simple Text Oriented Message Protocol) tầng trên WebSocket.
Auth	Không bắt buộc

Topic Subscribe — Dữ liệu cảm biến mới (/topic/sensor):


```
{
  "id": 1051,
  "temperature": 29.1,
  "humidity": 72.0,
  "light": 1340,
  "timestamp": "2026-09-30T14:40:02",
  "tempStatus": "Bình thường",
  "humidStatus": "Cảnh báo cao",
  "lightStatus": "Cảnh báo cao"
}
```


Topic Subscribe — Cập nhật trạng thái thiết bị (/topic/device-status):


```
{
  "id": 1,
  "deviceKey": "LED1",
  "name": "LED 01",
  "status": true,
  "online": true,
  "lastChanged": "2026-09-30T14:40:05"
}
```


Topic Subscribe — Cập nhật ActionLog (/topic/action-log):

Broadcast ActionLogDTO để tránh LazyInitializationException khi serialize:


```
{
  "id": 45,
  "deviceKey": "LED1",
  "deviceName": "LED 01",
  "deviceType": "light",
  "username": "admin",
  "userFullName": "Tran Ngoc Anh",
  "action": "ON",
  "status": "success",
```

```
  "timestamp": "2026-09-30T14:40:05"
}
```

Luồng hoạt động WebSocket:

1. Frontend khởi tạo kết nối SockJS → STOMP tới ws://localhost:8080/ws
2. Frontend Subscribe vào /topic/sensor, /topic/device-status và /topic/action-log
3. ESP32 gửi dữ liệu → MQTT Broker → Backend nhận qua MqttService
4. Backend lưu DB → Broadcast STOMP đến tất cả Client đang Subscribe
5. Frontend React nhận event → cập nhật state → re-render biểu đồ & thẻ số liệu

1.7.10 API-10 - Xem hồ sơ cá nhân

Thuộc tính	Nội dung
Method	GET
Endpoint	/api/auth/profile/{username}
Mô tả	Tra cứu và hiển thị toàn bộ thông tin cá nhân của người dùng hiện tại, bao gồm các liên kết đến tài nguyên dự án.
Auth	Yêu cầu đăng nhập
Bảng DB liên quan	users

Response Body (200 OK):

```
{
  "success": true,
  "message": "Thành công",
  "data": {
    "id": 1,
    "username": "admin",
    "fullName": "Phạm Mai Linh",
    "email": "B23DCCN488@ptit.edu.vn",
    "studentId": "B23DCCN488",
    "classRoom": "D23CQCN02-B",
    "githubUrl": "https://github.com/...",
    "figmaUrl": "https://figma.com/...",
    "postmanUrl": "https://postman.com/...",
    "reportUrl": "https://docs.google.com/...",
```

```
    "avatarUrl": "https://..."
  }
}
```

1.7.11 API-11 - Cập nhật hồ sơ

Thuộc tính	Nội dung
Method	PUT
Endpoint	/api/auth/profile/{id}
Mô tả	Cập nhật thông tin cá nhân và các đường dẫn URL tài liệu tham khảo của người dùng.
Auth	Yêu cầu đăng nhập
Bảng DB liên quan	users

Request Body:

```
{
  "username": "admin",
  "fullName": "Phạm Mai Linh",
  "email": "B23DCCN488@ptit.edu.vn",
  "studentId": "B23DCCN488",
  "classRoom": "D23CQCN02-B",
  "githubUrl": "https://github.com/linh/iot",
  "figmaUrl": "https://figma.com/file/...",
  "postmanUrl": "https://postman.com/...",
  "reportUrl": "https://docs.google.com/...",
  "avatarUrl": "https://..."
}
```

Response Body (200 OK):

```
{
  "success": true,
  "message": "Cập nhật thành công",
  "data": {
    "id": 1,
    "username": "admin",
    "fullName": "Phạm Mai Linh",
    "email": "B23DCCN488@ptit.edu.vn",
    "studentId": "B23DCCN488",
```

```
        "classRoom": "D23CQCN02-B",
        "githubUrl": "https://github.com/linh/iot",
        "figmaUrl": "https://figma.com/file/...",
        "postmanUrl": "https://postman.com/...",
        "reportUrl": "https://docs.google.com/...",
        "avatarUrl": "https://..."
      }
    }
```

CHƯƠNG 2. KẾT QUẢ VÀ ĐÁNH GIÁ THỰC NGHIỆM

2.1 Cấu trúc thư mục dự án



Hình 3 Cấu trúc thư mục

2.2 Kết quả triển khai phần cứng

Phần cứng của hệ thống đã được chế tạo và lắp ráp hoàn chỉnh trên bo mạch cắm (breadboard) thử nghiệm với độ tin cậy và thẩm mỹ cao:

- Bộ vi điều khiển: ESP32-WROOM-32 được nạp mã nguồn C++ phát triển trên nền tảng Arduino IDE.
- Khối cảm biến: Cảm biến DHT11 và quang trở LDR hoạt động ổn định, đọc dữ liệu định kỳ chuẩn xác.
- Khối hiển thị & chấp hành: 3 đèn LED đỏ, vàng, xanh phản ứng tức thì khi nhận được tín hiệu kích hoạt từ các chân GPIO 5, 18, 19.

- Kết nối mạng & Thời gian thực: ESP32 tự động quét và kết nối thành công vào mạng Wi-Fi, kết nối đồng bộ thành công với máy chủ NTP pool.ntp.org và gán nhãn thời gian thực chuẩn GMT+7 vào từng gói tin.

2.3 Kết quả triển khai Broker và dịch vụ Backend

- Broker Mosquitto: Chạy ổn định ở cổng 1884, tiếp nhận thông lượng gói tin đều đặn mỗi 2 giây từ phần cứng và cho phép Spring Boot Client subscribe lấy tin tức thời.
- Kiểm thử giả lập dòng dữ liệu với Mosquitto CLI:
    - Lệnh lắng nghe dữ liệu cảm biến:

mosquitto_sub -h localhost -p 1884 -t "iot/data_sensor" -u "PhamMaiLinh" -P "B23DCCN488"

    - Lệnh giả lập điều khiển bật đèn:

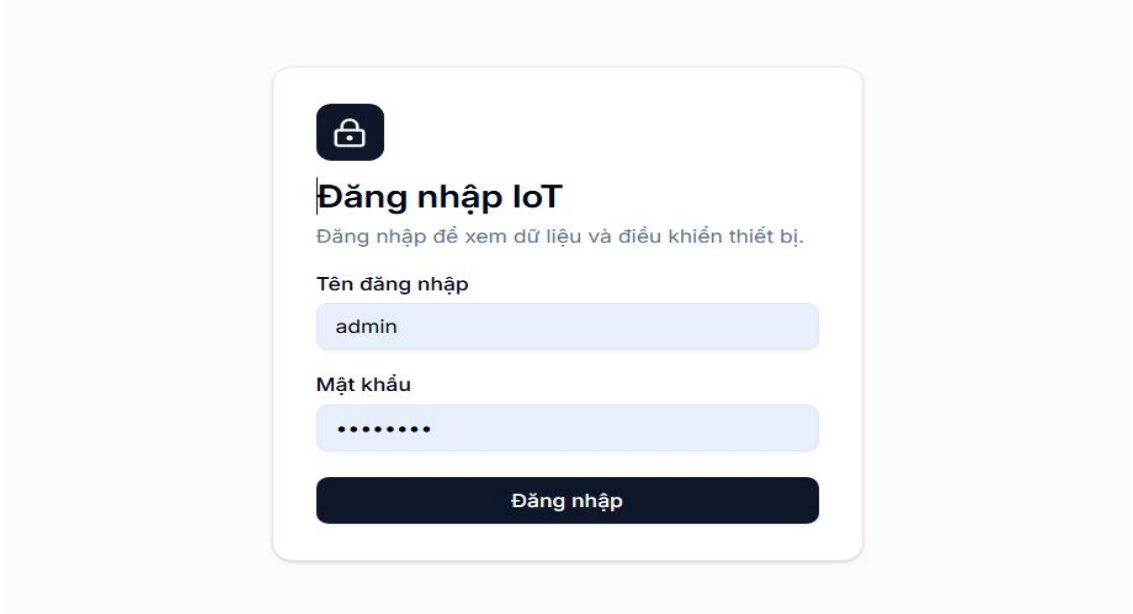
mosquitto_pub -h localhost -p 1884 -t "iot/devices_control" -u "PhamMaiLinh" -P "B23DCCN488" -m "LED1_ON"

- Máy chủ Spring Boot:
    - Khởi động trên cổng 8080, nạp thành công mô hình dữ liệu Hibernate vào cơ sở dữ liệu iot_db.
    - DataInitializer tự động khởi tạo: tài khoản admin (BCrypt admin123), 3 thiết bị LED, 1 cảm biến DHT11_LDR. Migration 2 giai đoạn: (1) findByTempStatusIsNull() — cập nhật status + gán FK sensor; (2) findBySensorIsNull() — gán FK sensor cho bản ghi chưa có. File migration.sql hỗ trợ thêm các FK constraints thủ công (fk_datasensor_sensor, fk_actionlog_device, fk_actionlog_user) sau khi đã làm sạch dữ liệu orphan.
    - Tích hợp thành công logic @PrePersist tự động phân loại trạng thái vi khí hậu ("Bình thường", "Cảnh báo cao", "Cảnh báo thấp") mà không làm nghẽn luồng xử lý.
    - DataSensorRepository.findWithFilters(...) xử lý linh hoạt việc tìm kiếm theo dải giá trị min/max của từng cảm biến và khoảng thời gian tùy biến.
    - WebSocket STOMP Server Broadcast thành công trên 3 topic: /topic/sensor, /topic/device-status, /topic/action-log; giao diện Frontend cập nhật không cần tải lại trang.

2.4 Kết quả xây dựng giao diện người dùng Frontend

Ứng dụng web Frontend đã hoàn thành 100% các yêu cầu đề ra với phong cách thiết kế hiện đại, tinh tế:

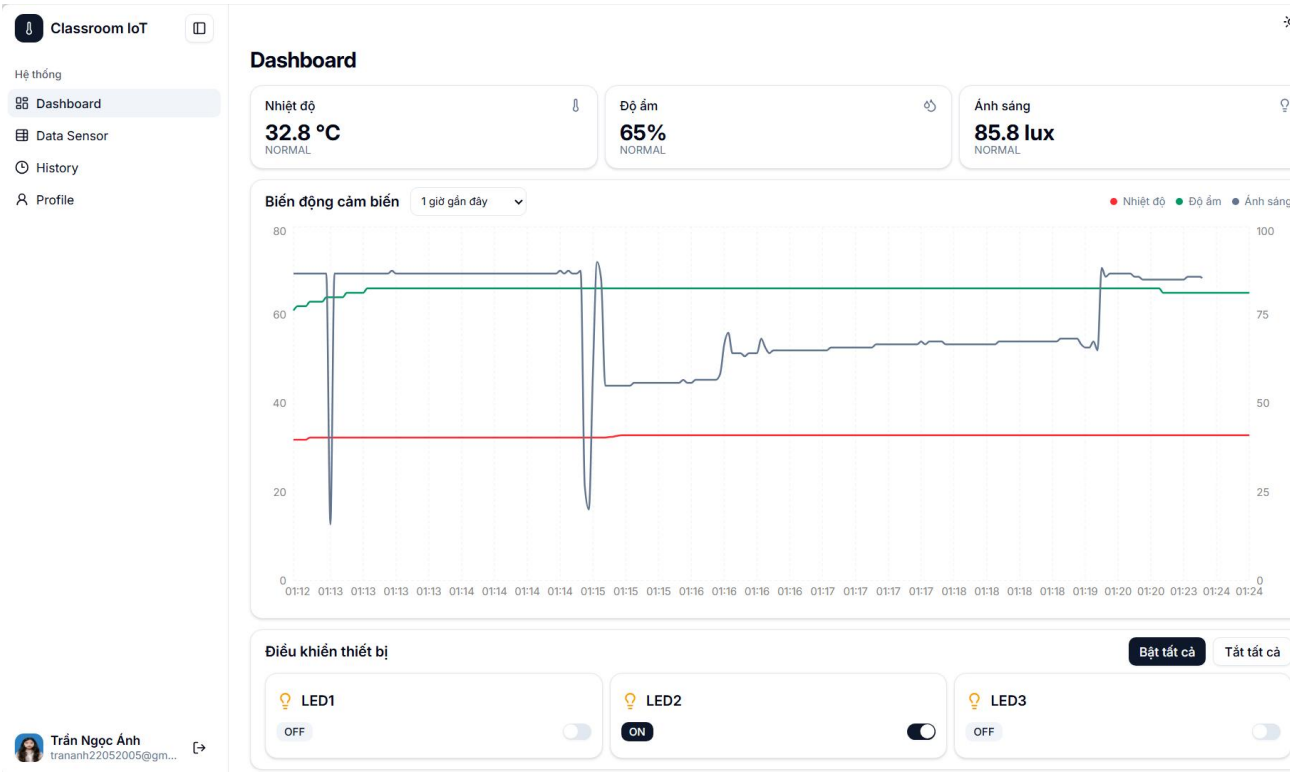
1. Màn hình Đăng nhập (Login Screen): Thiết kế form đăng nhập thanh lịch, kiểm tra tính hợp lệ của trường thông tin và hiển thị thông báo lỗi trực quan.



Hình 4 Màn hình Đăng nhập

2. Trang Giám sát Tổng quan (Dashboard):

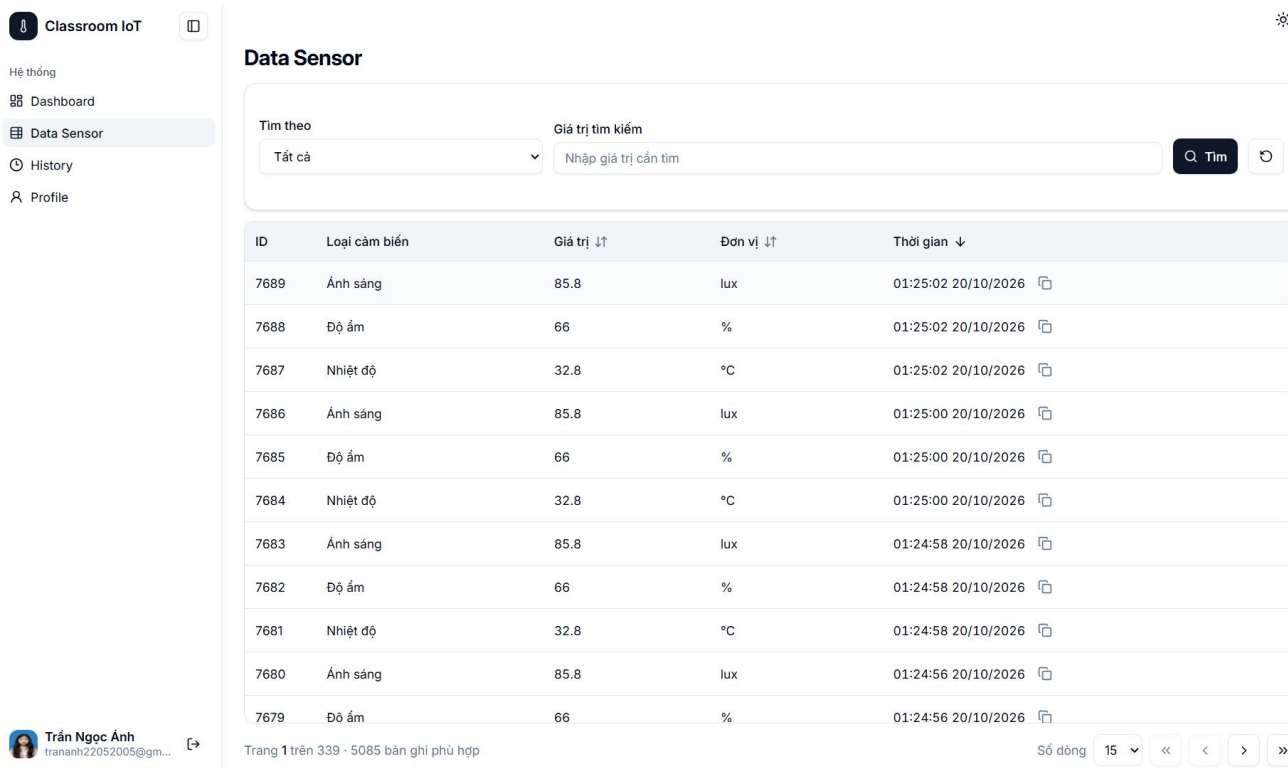
- 3 Card hiển thị thông số: Nhiệt độ, Độ ẩm, Cường độ ánh sáng với màu sắc phân biệt, icon sinh động và Badge nhãn trạng thái cảnh báo ("Bình thường", "Cảnh báo cao", "Cảnh báo thấp").
- Biểu đồ đường (Line Chart) đa trục, tự động co giãn và cập nhật tức thời khi có gói tin mới từ phần cứng.



Hình 5 Màn hình Dashboard

3. Màn hình Lịch sử Cảm biến (DataSensor History):

- Bảng hiển thị có đầy đủ mã ID tự tăng, tên cảm biến, giá trị, thời gian và Badge cảnh báo màu sắc tương ứng.
- Thanh tìm kiếm hỗ trợ chuyển đổi chế độ lọc: Tất cả, Nhiệt độ, Độ ẩm, Ánh sáng, Thời gian.
- Hỗ trợ tìm kiếm theo dải giá trị (ví dụ: 20 - 30) và tìm kiếm thời gian linh hoạt yyyy/MM/dd HH:mm:ss.



Hình 6 Màn hình Lịch sử cảm biến

4. Màn hình Lịch sử Bật/Tắt (Action History):

- Hiển thị chi tiết từng lần can thiệp vào thiết bị kèm tài khoản thao tác (user_action).
- Bộ lọc 4 tiêu chí: theo thiết bị, theo hành động, theo trạng thái và theo thời gian.